

Distributed consensus

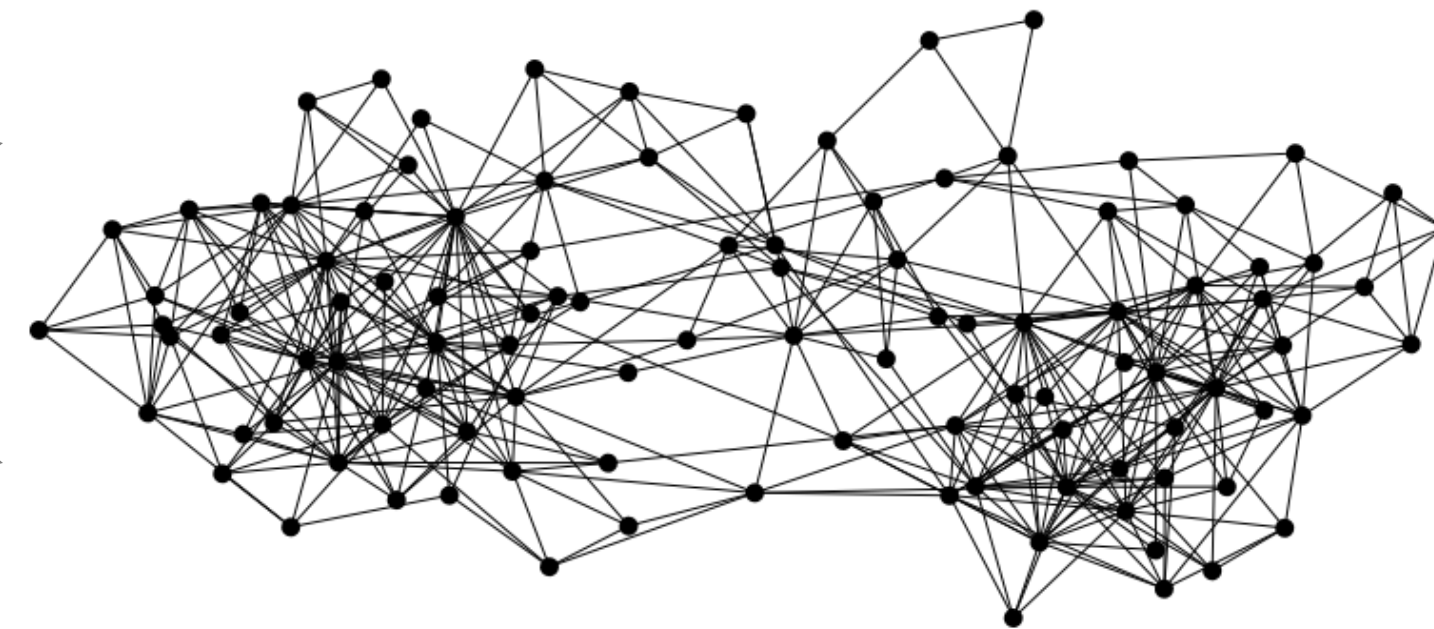


Bitcoin is a peer-to-peer system

When Alice wants to pay Bob:
she broadcasts the transaction to all Bitcoin
nodes



signed by Alice
Pay to $pk_{\text{Bob}} : H()$



Note: Bob's computer is not in the picture

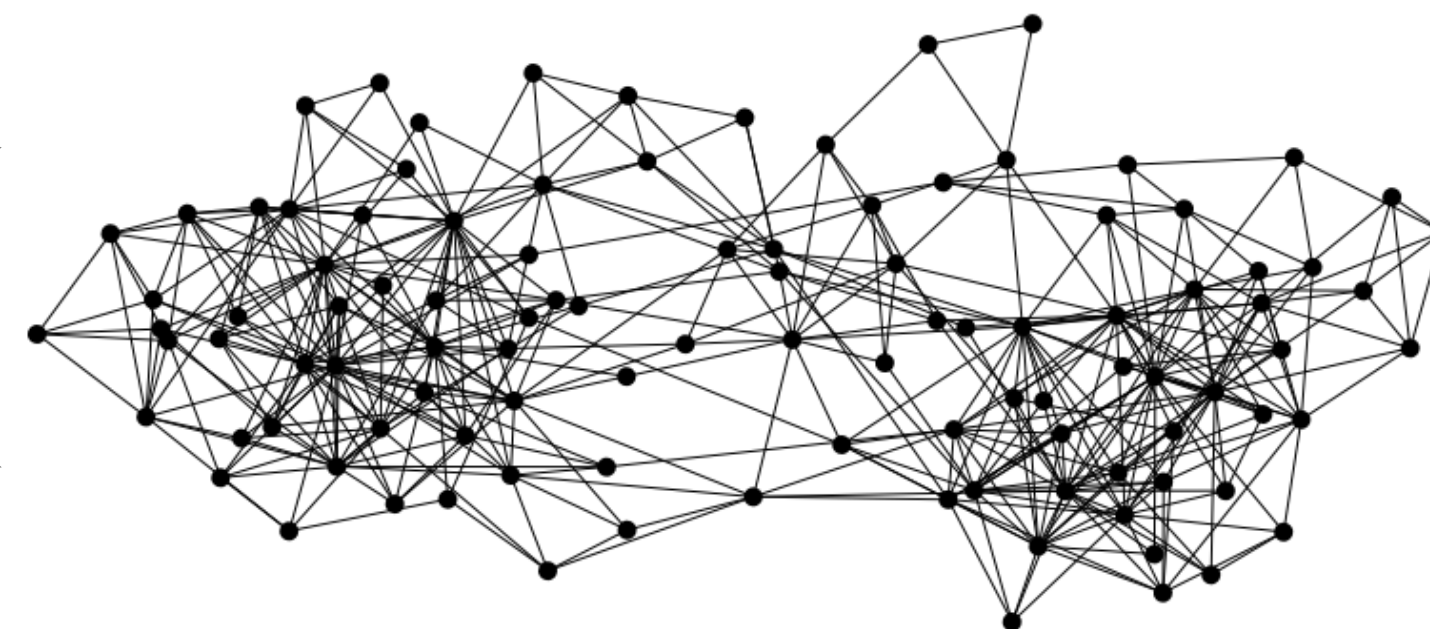
Bitcoin is a peer-to-peer system

This network is a fill/flood style P2P network:
all nodes perform basic validation, then relay
to their peers

This introduces bootstrapping, spam and DoS
problems, which are dealt with through “seeders”
and “reputation” scores



signed by Alice
Pay to $pk_{Bob} : H()$



CVE-2018-17145: Bitcoin Inventory Out-of-Memory Denial-of-Service Attack

Braydon Fuller and Javed Khan

September 9th, 2020

Abstract

This paper describes an easily exploitable uncontrolled memory resource consumption denial-of-service vulnerability that existed in the peer-to-peer network code of three implementations of Bitcoin and other blockchains including Litecoin, Namecoin and Decred.

1 Attack Overview

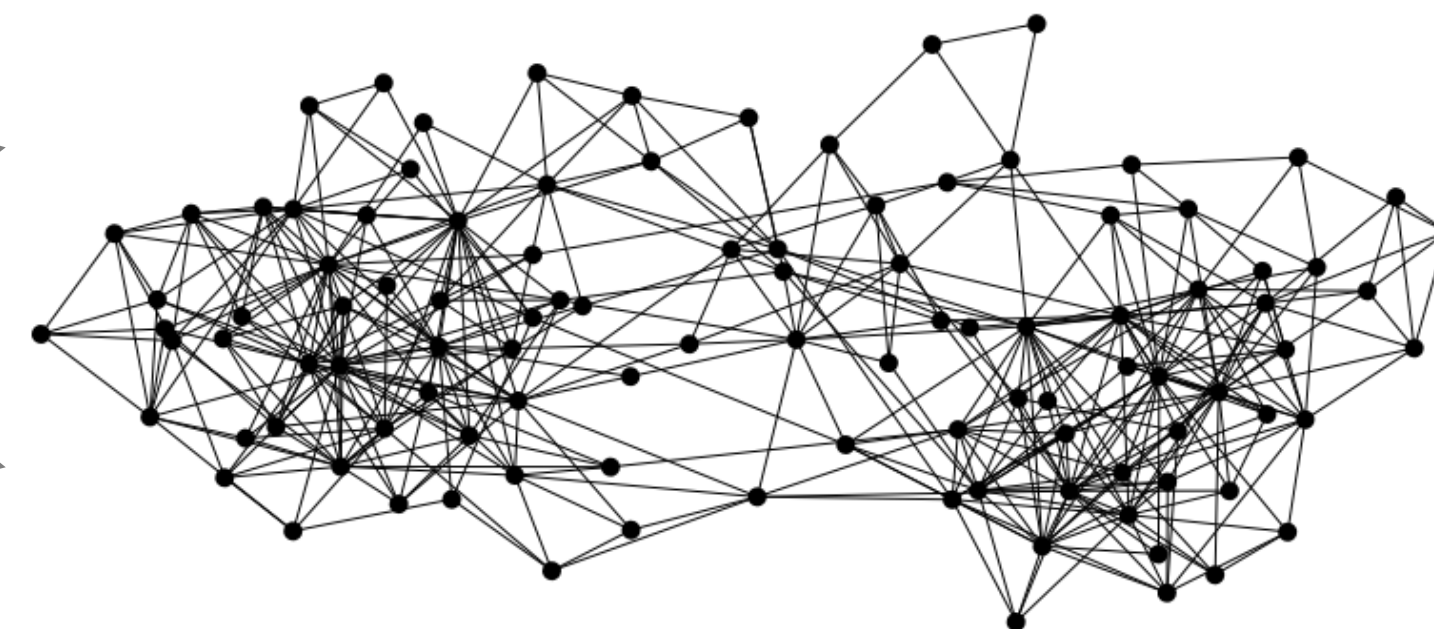
There was an uncontrolled resource consumption and out-of-memory (OOM) vulnerability that could have been easily exploited in a denial-of-service (DoS/D-DoS) attack against many Bitcoin, Litecoin, Namecoin and Decred nodes by

Why aren't we done here?

Why can't we just trust this system to eliminate invalid blocks, and give everyone a robust view of the Tx history?



signed by Alice
Pay to pk_{Bob} : $H(\quad)$



Bitcoin's key challenge

Key technical challenge of decentralized e-cash: distributed consensus

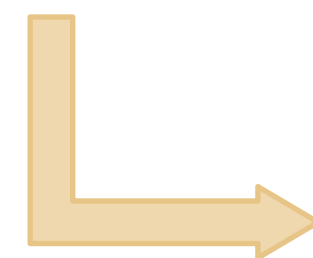
or: how do all of these nodes agree on an ordered history of transactions?

Why consensus protocols?

Traditional motivation: reliability in distributed systems

Distributed key-value store enables various applications:

DNS, public key directory, stock trades ...



Good targets for Altcoins!

Defining distributed consensus

The protocol terminates and all honest nodes decide on the same **value**

This value must have been proposed by some honest node

Defining distributed consensus

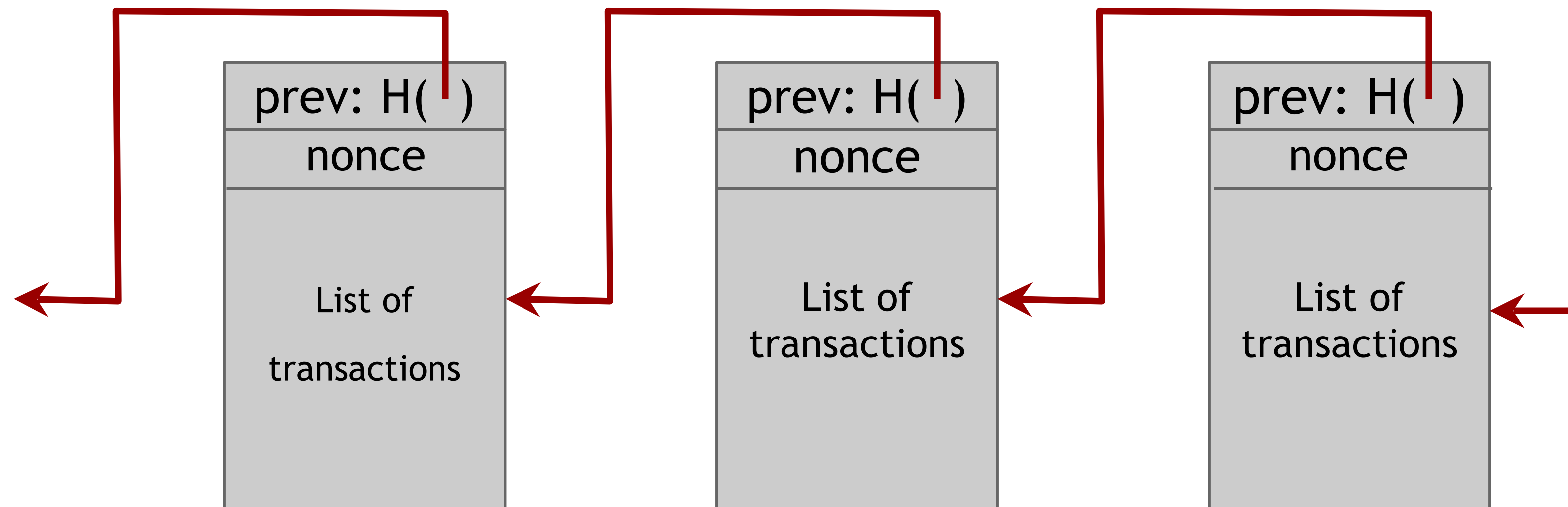
Q: What is this “value”
in Bitcoin?

The protocol terminates and all honest nodes
decide on the same **value**

This value must have been proposed by some
honest node

A: In Bitcoin, the value we want to agree on is the current state of the ledger. If we use a blockchain, that works out to this single hash

$H()$

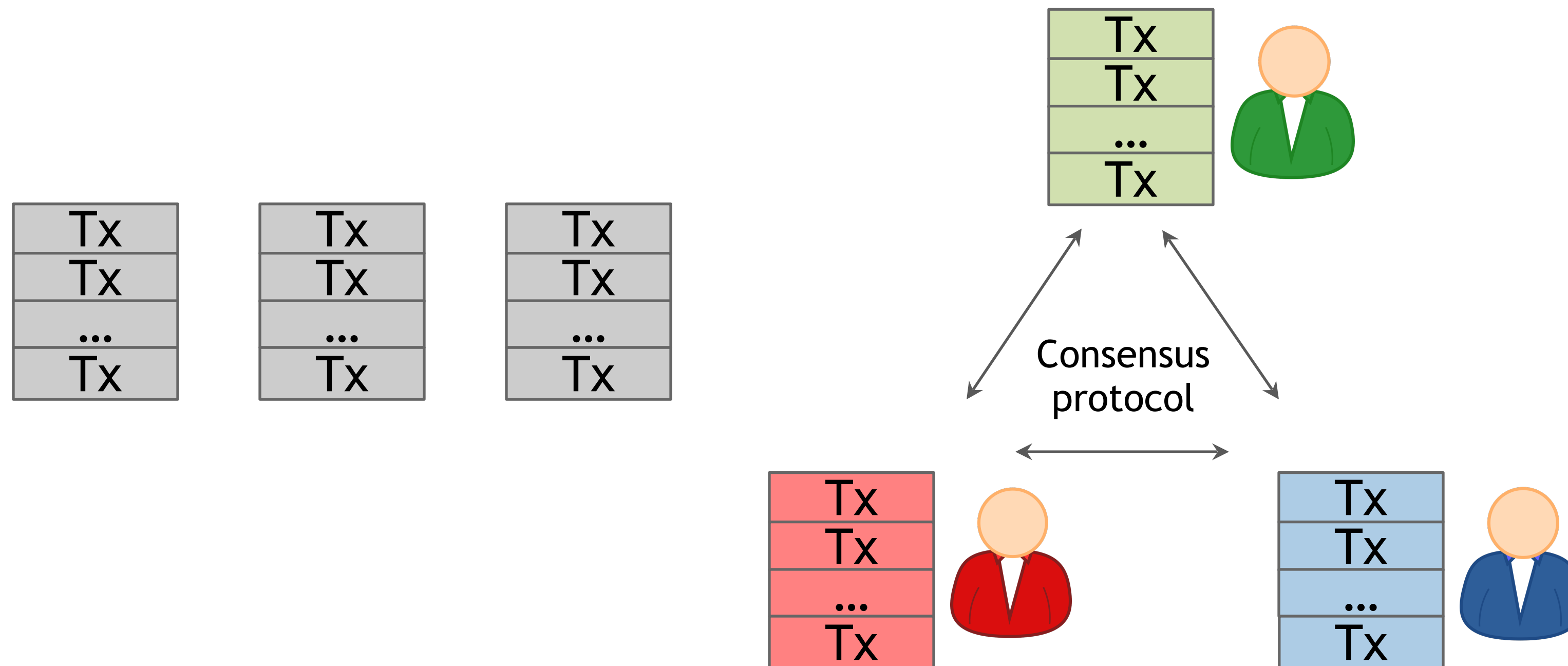


How consensus could work in Bitcoin

At any given time:

- All nodes have a sequence of blocks of transactions they've reached consensus on
- (Blocks are also distributed via p2p network)
- Each node has a set of outstanding transactions it's heard about

How consensus could work in Bitcoin



OK to select any valid block, even if proposed by only one node

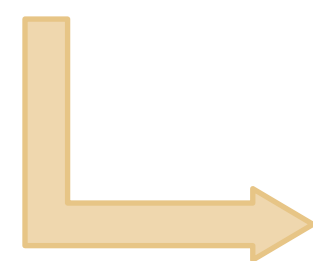
Why consensus is hard

Nodes may crash

Nodes may be malicious

Network is imperfect

- Not all pairs of nodes connected
- Faults in network (“partitioning”)
- Latency



No notion of global time

Defining distributed consensus

The protocol terminates and all honest nodes decide on the same value (history)

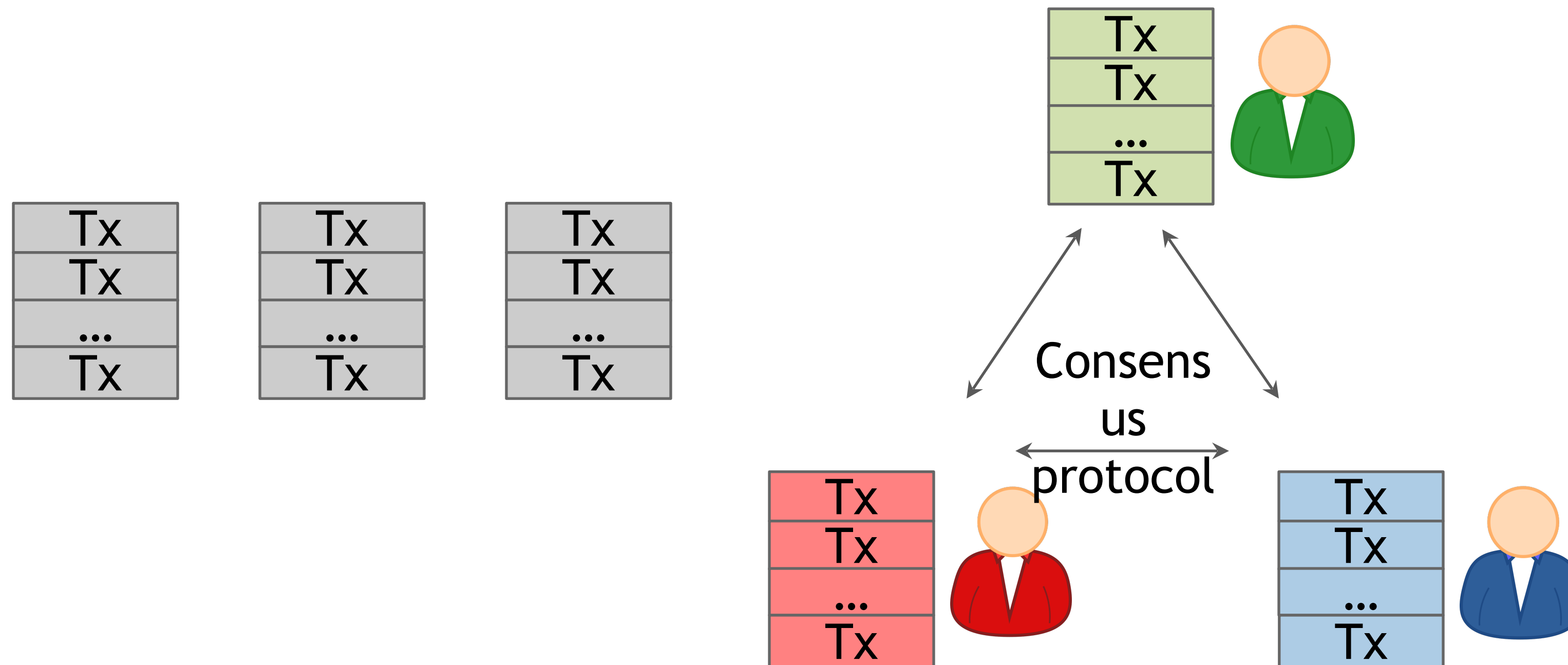
This value must have been proposed by some honest node

How consensus could work in Bitcoin

At any given time:

- All nodes have a sequence of blocks of transactions they've reached consensus on
- Each node has a set of outstanding transactions it's heard about

How consensus could work in Bitcoin



OK to select any valid block, even if proposed by only one node

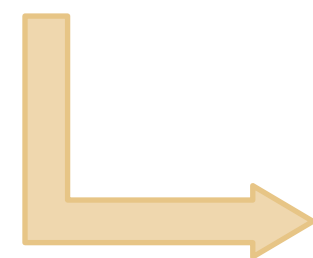
Why consensus is hard

Nodes may crash

Nodes may be malicious

Network is imperfect

- Not all pairs of nodes connected
- Faults in network
- Latency



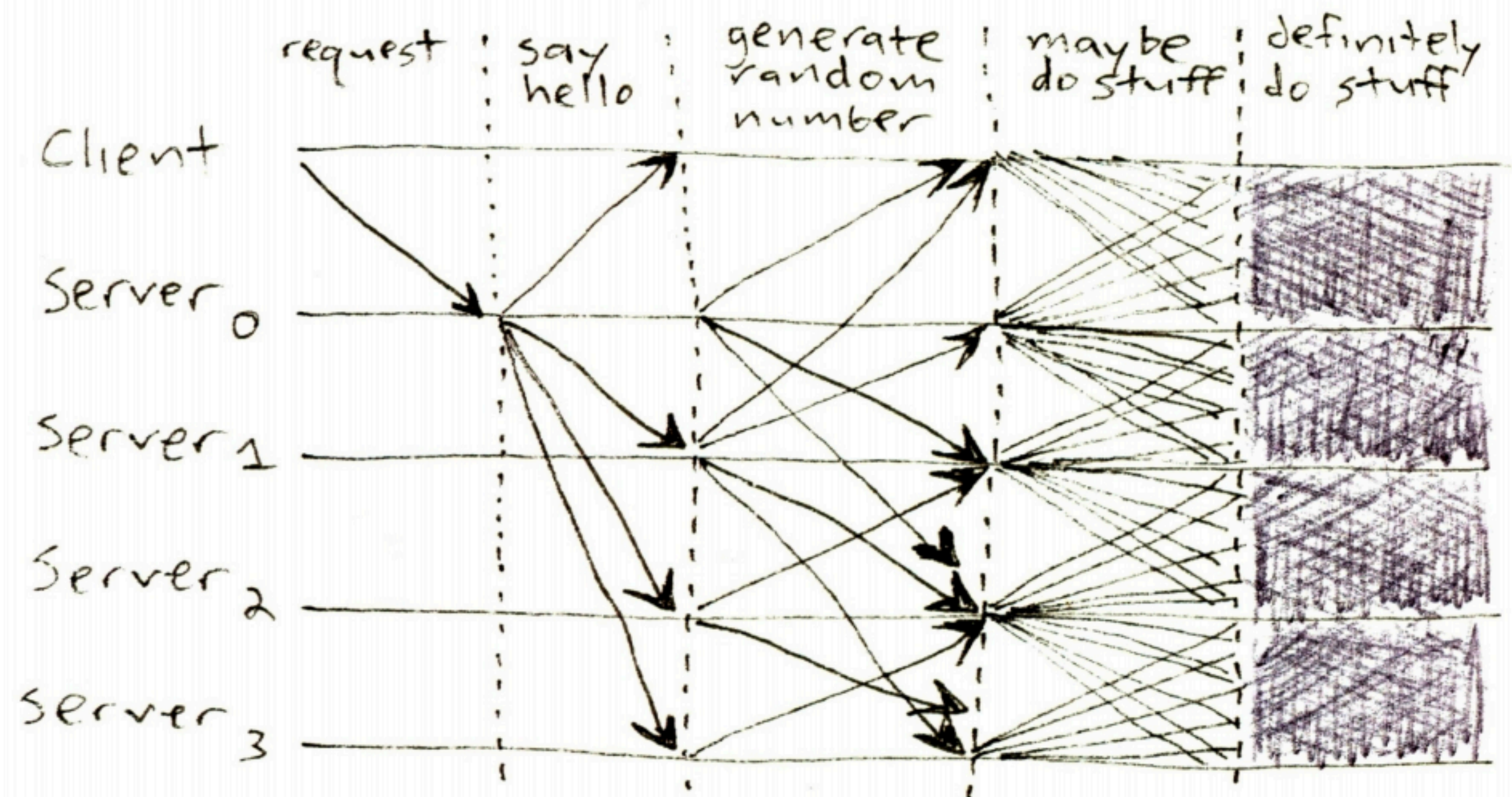
No notion of global time

Many impossibility results

- Impossible without $2/3$ honest majority [Pease, Shostak, Lamport'80]
- Impossible with a single faulty node, in the fully asynchronous setting, with deterministic nodes [Fischer-Lynch-Paterson'85]

Why do these results matter?

- Because without node identities, an attacker could easily crash these networks by impersonating many nodes (“Sybil attack”)
- Because synchronicity is hard



Some positive results

Example: Paxos [Lamport]

Never produces inconsistent result, but can (rarely) get stuck

Understanding impossibility results

These results say more about the model than about the problem

The models were developed to study systems like distributed databases

Bitcoin consensus: theory & practice

- Bitcoin consensus: initially, seemed to work better in practice than in theory
- Theory has been steadily catching up to explain why Bitcoin consensus works [e.g., Garay-Kiayias-Leonardos'15, Pass-Shelat-Shi'17, Garay-Kiayias-Leonardos'17,...]
- Theory is important, can help predict unforeseen attacks

Some things Bitcoin does differently

Introduces incentives

- Possible only because it's a currency!

Embraces randomness

- Does away with the notion of a specific end-point
- Consensus happens over long time scales — about 1 hour

Consensus without identity: the blockchain



Why identity?

Pragmatic: some protocols need node IDs

Security: assume less than 50%
malicious

Why don't Bitcoin nodes have identities?

Identity is hard in a P2P system —
Sybil attack

Pseudonymity is a goal of Bitcoin

Weaker assumption: select random node

Analogy: lottery or raffle

When tracking & verifying identities is hard, we give people tokens, tickets, etc.

Now we can pick a random ID & select that node

Key idea: implicit consensus

In each round, random node is picked

This node proposes the next block in the chain

Other nodes implicitly accept/reject this block

- by either extending it
- or ignoring it and extending chain from earlier block

Every block contains hash of the block it extends

Consensus algorithm (simplified)

1. New transactions are broadcast to all nodes
2. Each node collects new transactions into a block
3. In each round a random node gets to broadcast its block
4. Other nodes accept the block only if all transactions in it are valid (unspent, valid signatures) and it builds on a chain they already accept
5. Nodes express their acceptance of the block by including its hash in the next block they create

So how do we pick a
random node?

Resources & Consensus

- One computer can easily pretend to be many “nodes”, so simple random voting $\neq$ good
- But an observation: resources (e.g., hardware, storage, CPU, GPU, etc.) are much harder to fake
- Idea: make your probability of winning the vote proportional to your overall resources

Puzzles

- Early idea, proposed in the 90s: solve computational puzzles to prove CPU power
- Dwork & Naor (1992): use them to make spam email more expensive
- Back (1997): Hashcash, spam emails again
- Juels & Brainard (1999): use them to prevent DoS on web servers

Simple interactive puzzle



The web server accepts the GET iff $S = H(\text{Chal} \mid \text{Nonce})$
begins with “ D ” 0 bits

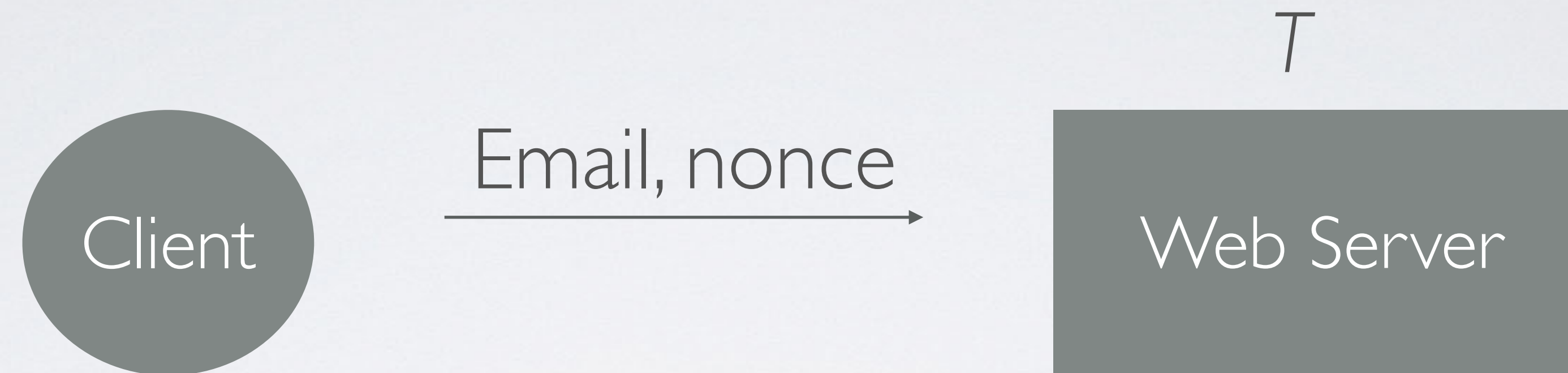
Simple interactive puzzle



Alternative formulation:

The web server accepts iff $H(\text{Chal} \mid \text{nonce}) < T$

Spam/Hashcash



The web server accepts iff $H(\text{Email} \mid \text{nonce}) < T$
and the email hasn't been seen before

Bitcoin PoW

The web server accepts iff $H(\text{Email} \mid \text{nonce}) < T$
and the email hasn't been seen before

Key idea: implicit consensus

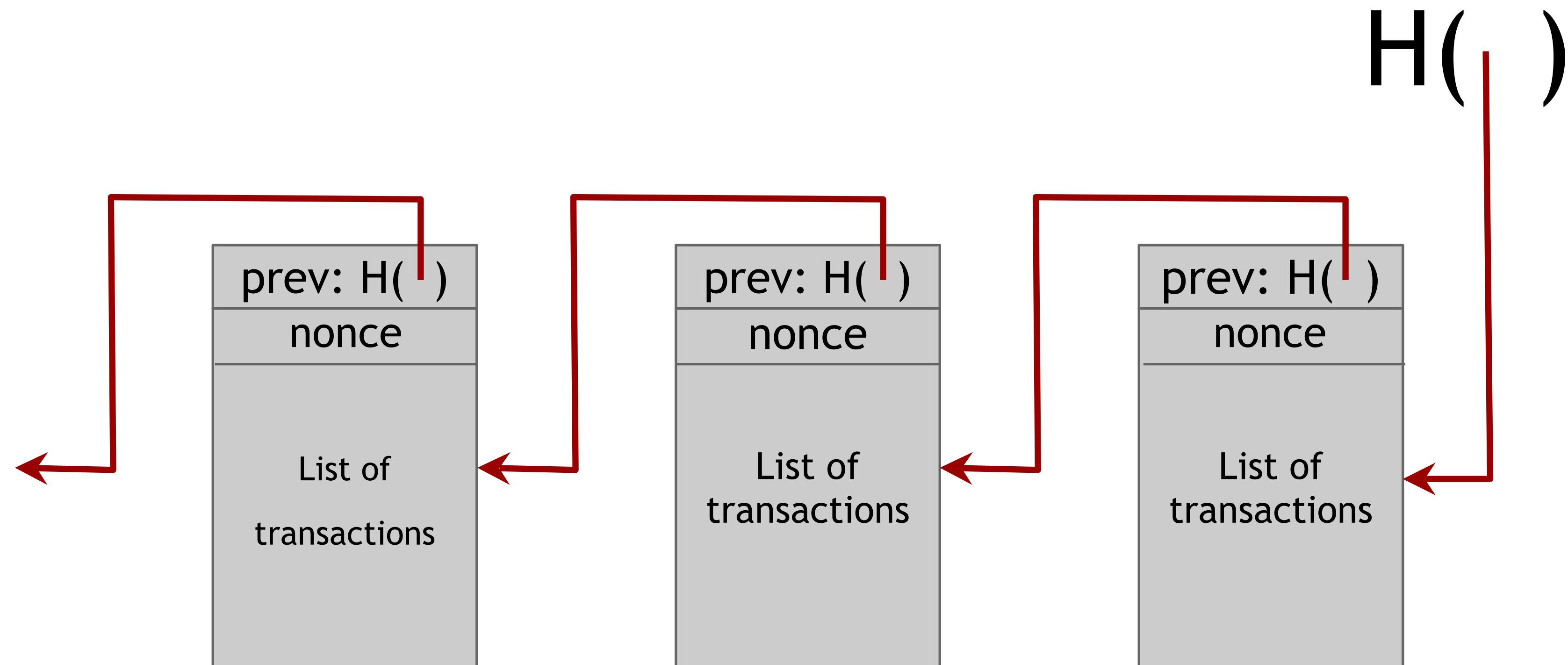
In each round, all nodes compete to solve a puzzle

The winner proposes the next block in the chain and sends out their solution along with it

Other nodes implicitly accept/reject this block

- by either extending it
- or ignoring it and extending chain from earlier block

Every block contains hash of the block it extends



What's the puzzle?

The puzzle is simply the hash of the new block, which must be chained off the previous block

I.e., find a **nonce** such that $H(\text{block} \mid \text{nonce}) < T$ for some T

The winner proposes the next block in the chain and sends out their nonce

Other nodes implicitly accept/reject this block

- by either extending it
- or ignoring it and extending chain from earlier block

What happens if nodes ignore the block?

Where do we get T ?

The value T is called the “block difficulty target”.
It’s adjustable.

Ideas:

- Choose T once at the start, keep fixed
- Change T from time to time

What are the impacts of these choices?